Ders 2 Genişletilmiş Notu: Bilgi Sistemi Türleri, Bilgi Sisteminin Tarafları, Geliştirme Süreci ve Süreç Modelleri

Ders 2 Genişletilmiş Notu: Bilgi Sistemi Türleri, Bilgi Sisteminin Tarafları, Geliştirme Süreci ve Süreç Modelleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı ve Bilgi Sistemi Tanımının Hatırlanması
- 2. Bilgi Sistemi Türleri ve Seviyeleri
- 3. Türlere Birim Birim Örnekler
- 4. Türlerin Ayrıntıları
- 5. Türlerin Karşılaştırılması ve Aralarındaki Etkileşim
- 6. Karma Sistemler
- 7. Bilgi Sisteminin Tarafları
- 8. Bilgi Sistemi Geliştirme Süreci
- 9. Süreç Modelleri
- 10. Ders Sonundaki Sorular

1. Dersin Planı ve Bilgi Sistemi Tanımının Hatırlanması

Hoca dersi üç başlık üzerine kurdu. Birincisi bilgi sistemi türleridir: bir bilgi sistemi türü nedir, ne işe yarar, türleri neden ayırırız, aralarında ne gibi farklar vardır, birbirleriyle nasıl etkileşirler? İkincisi bir bilgi sisteminin taraflarıdır: taraflar kimlerdir, rolleri ve sorumlulukları nelerdir? Üçüncüsü de bir bilgi sistemini geliştirmek için izlenmesi gereken yol haritasıdır: bir proje hangi safhalardan geçerek doğru ve düzgün bir şekilde üretilebilir? Bu son başlıkta farklı yaklaşımlar kısaca tanıtılacak, asıl durulacak olan ise temel proje akış süreci olacaktır.

İlk derste verilen tanım şöyle hatırlatıldı. Elimizde belli problemler vardır. Bu problemleri analiz etmek, çeşitli işlerin, süreçlerin ve işlemlerin yürütülmesini sağlamak, yeni ürünler oluşturmak, yönetimin faaliyetlerine destek olmak, karmaşık nesneleri canlandırmak, işletmek ve kontrol etmek, karmaşık nesne ve yapıların koordinasyonunu sağlamak, böylece karar almaya destek olmak gibi amaçlarımız olabilir. Bu amaçları yerine getirmek üzere; girdileri olan, bir arada uyumlu çalışan bileşenlerden ve alt sistemlerden oluşan, dış sistemlerle ara birimler aracılığıyla iş birliği yapan, sınırları belli ve çıktı üreten yapılara **bilgi sistemi** adını veriyoruz.

2. Bilgi Sistemi Türleri ve Seviyeleri

2.1 Türlere Neden Ayırıyoruz?

Tanımdaki amaçlar birbirinden farklıdır ve bilgi sistemi türlerini oluşturan da bu farklılıktır. Belli işlerin yürütülmesini sağlayan sistemler bir sınıfta, yönetim faaliyetlerini destekleyenler başka bir sınıfta, bilgiyi kullanarak iş yapanlar ya da karmaşık yapıların koordinasyonunu sağ-

layanlar ayrı bir sınıfta, karar almaya destek olan çıktılar üretenler yine ayrı bir sınıfta toplanır. Yani bilgi sistemlerini amaçlarına göre türlere ayırırız.

Bütün bilgi sistemlerinde temel girdi veridir; işletme için gereken bilgiler de girdi olabilir. Temel işlemler bu veri ve bilgilerin toplanması, işlenmesi, sonradan kullanılmak üzere saklanması ve iş birliği içinde dağıtılmasıdır. Çıktılar ise kullanıma sunulan hizmetler ve üretilen bilgilerdir.

> **Hocanın vurgusu:** Bilgi sistemi türlerini literatürde genel olarak 6'ya ayırıyoruz. Bu ayrım yalnız bize özgü değildir, dünyadaki genel kaynaklarda da böyledir; ders boyunca aynı ayrım kullanılacaktır.

2.2 Seviyeler Piramidi

Hoca slaytta bir işletme ve birimlerini gösterdi: üretim, satış-pazarlama, finans, muhasebe, insan kaynakları. Bu birimler işletmeye göre değişebilir ve artabilir; stok ya da depo eklenebilir, üretim bölünebilir, bazı parçalar ara mamul olarak hazır alınabilir. Slayttaki beş birim yalnız basit bir örnektir. Türler ise bu birimlerin üzerine yerleşen, yukarı doğru daralan bir hiyerarşide seviyelere gruplanır.

- **İşlemsel seviye (pembe, en altta):** Temel işlerin yürütülmesini sağlar. Veriyi alır, işler, saklar, iletir ve gündelik işlerin bilgi sistemleri aracılığıyla yapılmasını sağlar. Bu seviyenin türü **işlemsel bilgi sistemidir** (Transaction Processing Systems, TPS).
- **Bilgi seviyesi (yeşil):** Bu sistemler de işlem yapar, ama yalnız ham veriyle ya da girdiyle çalışmaz. İşlemsel bilgi sistemlerinden beslenebilirler ve "bilgi" dediğimiz, hâlihazırda işlenmiş verileri de girdi olarak kullanabilirler. Bu seviyede iki tür bulunur:
  - **Bilgi tabanlı iş sistemleri** (Knowledge Based Systems ya da Knowledge Work Systems, KWS): İşlemsel sistemlerden temel farkları, bilgiyi de işleyerek daha anlamlı, daha özel, daha özet ve işimize yarayacak farklılaştırılmış çıktılar üretmeleridir.
  - **Ofis otomasyon sistemleri** (Office Automation Systems): Gündelik ofis ve işletme işlerinin yapılmasını, yürütülmesini ve birlikte çalışmayı sağlar. Ofis işlemlerini de yaptığı için pembe seviyeye yakındır; pembe ile yeşil arasında, biraz daha aşağıda düşünülmelidir. Gerektiğinde bir destek bilgi sistemi olarak da görülür. İçine bilgi de girmesi gerektiği için pembe değil yeşil seviyede sayılır.
- **Yönetim seviyesi (mavi):** Yönetim kademesine hizmet eder. İki tür vardır:
  - **Yönetim bilgi sistemleri** (Management Information Systems, MIS): Ara yönetimin ve birim yöneticilerinin işlerini yürütmesine yardımcı olur.
  - **Karar destek sistemleri** (Decision Support Systems, DSS): Yöneticilere hizmet eder. Şekilde biraz aşağıda durmasının sebebi yeşil seviyeyle de bağlantılı olmasıdır: bilgi tabanlı iş sistemlerini biraz barındırır, onlardan da yararlanarak karar vermeye destek olan faydalı çıktılar üretir.
- **Stratejik seviye (turuncu, en üstte): Üst yönetim bilgi sistemleri** (Executive Information Systems ya da Executive Support Systems) burada yer alır. Üst yönetim, hocanın "üç harfliler" dediği CEO, CIO, CFO, CTO gibi yöneticiler ile yönetim kurulu üyeleri ve yönetim kurulu başkanıdır. Bu sistemler, yöneticinin önündeki ekranda her şeyi özet biçimde görmesini ve projeksiyonlarla birlikte çıkarım yapmasını sağlayan araçlar sunar.

3. Türlere Birim Birim Örnekler

3.1 İşlemsel Seviye

Slayttaki birimlere göre işlemsel bilgi sistemi örnekleri şunlardır:

- **Satış ve pazarlama:** Sipariş oluşturma, izleme ve takip sistemi; müşteri etkileşimi sistemi.
- **Üretim:** Bir makineyi kontrol eden sistem; bir fabrikanın, tesisin ya da üretim hattının programlanmasını ve işletilmesini kontrol eden sistem. Örneğin ürün hatları doğru çalışıyor mu, makine düzgün çalışıyor mu diye bakar, hata olduğunda ilgili operatöre uyarı verir. Ham madde hareketlerini kontrol eden sistem de bu gruptadır.
- **Finans:** Nakit yönetimi için gelirleri, giderleri, girdileri ve çıktıları izleyen bir kasa uygulaması.

- **Muhasebe:** Genellikle bu iş için uzmanlaşmış hazır uygulamalar kullanılır. Bordro üretme, vergi hesaplama, borç ve alacak muhasebesi hesapları bu sistemlerde yapılır.
- **İnsan kaynakları:** Bütün çalışanların özlük bilgilerinin tutulduğu sistem. Hocanın öğrencilere yakın örneği öğrenci bilgi sistemidir: derslerin seçildiği, notların girildiği, ortalamaların ve başarı durumunun hesaplandığı, transkriptin üretildiği sistem.

3.2 Bilgi Seviyesi

Ofis otomasyonu tarafında Word, Excel, PowerPoint; Pages, Numbers, Keynote; Writer, Calc, Presenter gibi farklı ofis paketleri vardır. Bunlar hem işletmenin günlük işlerinin yürütülmesini hem de ekiplerin, hatta farklı işletmelerin eş zamanlı birlikte çalışabilmesini sağlar. Kelime işlem, doküman yönetimi, elektronik takvim uygulamaları da, hocanın o anda ders yaparken kullandığı elektronik toplantı ve video konferans uygulamaları da ofis otomasyon sistemi olarak sınıflandırılır.

Bilgi tabanlı iş sistemlerinde veri yanında bilgi de kullanılır. Bilgisayar destekli tasarım ve üretim (CAD, CAM) bunun örneğidir, çünkü burada saf veri yoktur; tasarım ve üretim için sisteme bilgi de sağlanmak zorundadır. Simülasyon uygulamaları ve sanal gerçeklik uygulamaları da bu türdendir. Hoca AR-VR gözlükleriyle yapılan eğitimleri örnek verdi: tıp eğitiminde bir ameliyatın kesme, biçme, dikme aşamalarını simülasyon ortamında öğreten uygulamalar ya da bir makineyi kullanmadan önce kullanımını sanal gerçeklik ortamında gösteren uygulamalar.

Bu türün bir başka örneği, Workstation adı verilen kuvvetli bilgisayarlarda çalışan, bilimsel yöntemlere dayanan uygulamalardır. Hocanın dediğine göre bunlar günümüzde genellikle Python ile yazılıyor.

Borsa ve deprem örneği. Demir çelik sektöründeki hisselerin fiyatlarını izlemek ve tahmin etmek istediğinizi düşünün. Borsa verileri ham veridir. Bir yandan da haberlerden bilgi alırsınız. Hocanın güncel örneği şuydu: Kahramanmaraş, Gaziantep, Hatay civarında büyük bir deprem oldu ve İskenderun'daki demir çelik fabrikası deprem ve tespit çalışmaları nedeniyle üretimi durdurduğunu duyurdu. Hocanın yaklaşık ifadesiyle bu, Türkiye'deki demir çelik üretiminin yüzde yirmi beş kadarının bir anda devre dışı kalması demektir. Burada veri yoktur ama bilgi vardır: Üretim yapamayan şirketin hisselerinde düşüş yaşanabilir. Bu bilgiyi modelleri koşturan sisteme verdiğinizde sistem veriyle bilgiyi harmanlar ve "fiyat şöyle olur, alırsak şöyle, satarsak böyle olur" gibi çıktılar üretir.

3.3 Karar Destek Sistemleri

Karar destek sistemleri veriyle, bilgiyle ve alttaki sistemlerin çıktılarıyla çalışarak yöneticinin karar vermesine destek olan çıktılar üretir. Temel nokta şudur: Yönetici sistemin çıktısını harfiyen uygulamak zorunda değildir. Çıktı yalnız bir öneridir, gerçek kararı yönetici verir.

Tencere-tava örneği. Tencere, tava, sahan, düdüklü tencere gibi ürünler ürettiğinizi ve son bir yılın ya da belli bir dönemin bütün sipariş, satış ve ham madde tedarik verilerini topladığınızı düşünün. "Önümüzdeki dönem hangisinden daha çok, kaç tane üretsek?" sorusuna karar destek sistemi, bu verilere bakarak "şu üründen şu kadar üretmek gerekir" gibi bir çıktı üretebilir. Siz yine de bunu kullanmak zorunda değilsiniz: "Kampanya yapacağım, tava üretimine öncelik vereceğim" diyerek kendi kararınıza devam edebilirsiniz. Sonrasında girdileri sisteme yeniden verip kararınızın doğru olup olmadığını da kontrol edebilirsiniz.

3.4 Uzman Sistemler

Karar destek sistemlerine ek olarak uzman sistemler vardır. Hocanın anlatımına göre uzman sistem daha kapalı bir kutu olarak çalışır: Duruma göre kararı sistem verir ve sonuç, başka sistemlerin çalışmasını ya da girdilerini üretecek şekilde otomatik olarak uygulanır. Kararı değiştirme ya da uymama imkânınız olmaz. Uzman sistemin ayrıntısı Bölüm 4.5'te gelecektir.

3.5 Yönetim ve Stratejik Seviye

Yönetim bilgi sistemleri orta kademe, üst kademeye yakın ve alt kademe yöneticilerin ve birim yöneticilerinin günlük işlerini yapabilmesi için araçlar sunar. Hocanın örnekleri, farklı girdi-

lerle yıllık bütçe oluşturan bir sistem ile bir yatırımın analizini yapan, sonuçlarını grafik olarak gösteren ve projeksiyon yapabilen bir sistemdir.

Üst yönetim bilgi sistemleri ise bir şirketin, ortaklığın, iştirakin ya da holdingin en tepe yöneticileri için yapılır (chief technology officer, chief information officer, chief executive officer, chief financial officer gibi) ve stratejik seviyede çalışır. Pazar analizi yapar ve şu sorulara yardımcı olur: Üretimimizi ya da ürün portföyümüzü nereye doğru değiştirelim? Nereye yatırım yapalım? İlerideki nakit akışımız ve kârımız nasıl değişebilir? Müşteri portföyümüzü nasıl değiştirmeli, segment değiştirmeli miyiz? Hocanın benzetmesiyle bunlar holding patronlarının ekranları gibidir: Olup biteni izlerler, ayrıca yalnız kendi sektörünü değil farklı sektörleri de içine alacak şekilde yatırımlarını yönetmelerini sağlarlar.

4. Türlerin Ayrıntıları

4.1 İşlemsel Bilgi Sistemleri

İşlemsel bilgi sistemleri temel ve günlük işlerin yapılmasını sağlar, başka sistem türlerine giriş sağlar ve onların temelini oluşturur. Hoca bunları "işçi sınıfı" gibi düşünmemizi istedi, çünkü en çok işi bu sistemler yapar.

Bu sistemlerde önce veri toplanır, hazırlanır ve sisteme girilir. Veri girişi otomatik, yarı otomatik, başka sistemlerden gelen ya da elle girilen türde olabilir; bu yüzden hazırlama ve düzeltme operasyonları gerekebilir. Sistem veriyi işler, saklar, sınıflar, bakımını yapar, değiştirir, günceller ve istendiğinde geri çağırır.

Turnike örneği. Kampüse geldiniz ve kapıda kartınızı turnikeye bastınız.

1. Kartın içinde bir kimlik (ID) numarası, turnikede ise kart okuyucu vardır. Numara RFID teknolojisiyle temassız olarak okunur.
2. Bilgi bir ağ protokolü üzerinden sunucu bilgisayara iletilir: "Şu tarihte, şu saatte, şu numaralı kart, şu numaralı turnikeden giriş yönünde okundu." Bu bir kayıttır.
3. Kontroller yapılır: Geçişe izin verilecek mi? Kişinin girmesi yasaksa ya da süresi geçmişse turnike kırmızı yanmalı, geçit vermemeli, belki ötmelidir.
4. İzin verilirse "bu kişi bu tarihte bu turnikeden girdi" kaydı oluşturulur ve kişinin kampüs içinde olduğu bir veri tablosunda tutulur.
5. Sorgular cevaplanır: "Şu an içeride mi?" "Şu tarihte girdi mi?" "Ne zaman girdi?" Ayrıca "7 Mart 2023 Salı sabah 9 ile 11 arasında B kapısından yapılan bütün girişler" gibi bir sorgunun raporu üretilebilmelidir.

4.2 Yönetim Bilgi Sistemleri

Yönetim bilgi sistemleri alttaki sistemlerin verilerini kullanır. Hibrit, karmaşık ve çapraz raporlar ile analizler üretir; bu çıktıları bir karar destek sistemine ya da yöneticiye verir.

Bu işler için farklı işlemsel bilgi sistemleriyle etkileşim ve entegrasyon içinde bir uygulama tabanı, veri tabanı yönetim sistemi ve veri tabanına erişim gerekir. Bazen bir **veri ambarına** (data warehouse, DWH) da ihtiyaç duyulur. Hoca, henüz veri tabanı konusuna bile gelinmediğini söyleyerek kavramı kabaca tanımladı: Çok büyük ve yüksek hacimli verilerin farklı projeksiyonlardaki özetleriyle birlikte üretilip saklandığı ve daha esnek sorgular yapılabilen bir ortam.

Sistemin sağladığı fayda yönetim kademesine göre değişir:

- **Alt yönetici için:** İşlemlerin doğru yapılıp yapılmadığını ve güncelliğini kontrol etmek.
- **Orta yönetim için:** Alttaki birimlerin kaynakları kontrollü biçimde paylaşmasını yönetmek. Örneğin bir üretim hattını farklı üretim birimleri kullanıyorsa hangisinin neyi ne zaman kullanacağının planlanması, yürütülmesi ve takibi bu sistem üzerinden yapılabilir.
- **Üst yönetim için:** Stratejik hedefler ve planlamalar için çıktı üretmek.

İşlevsel açıdan da birimler arasında erişilmesi gereken bilgilerin yöneticiler aracılığıyla iletilmesini sağlar.

Mobil uygulama şirketi örneği. Kendi uygulamalarını da dış kaynaklı (outsource) uygulamaları da yazan bir şirket düşünün. iOS, Android gibi ortamlar için yazılım birimleri, satış ve pazarlama ekipleri, bir de Ar-Ge birimi var. Ar-Ge, uygulamaların indirme sayılarını izleyip düşüş olduğunda uyarı üreten bir proje yaratıyor. Bu proje aslında satış ve pazarlamanın işine yarar; dolayısıyla onlara sunulmalı, kullanımlarına açılmalı, onlardan da bilgi alınmalıdır. Birimler arasındaki bilgi farklarını dengeleyen, aradaki trafiği yöneten ve istenen veriyi öbür tarafa aktaran uygulamalar da yönetim bilgi sistemi olarak sınıflandırılır.

4.3 Ofis Otomasyon Sistemleri

Ofis otomasyon sistemleri Bölüm 3.2'de anlatıldı. Hoca burada yalnız telekonferans uygulamalarını yineledi: Birlikte ve verimli çalışmayı sağladıkları için bunlar da bu türe girer.

4.4 Karar Destek Sistemleri

Karar destek sistemlerine veri ve başka sistemlerden gelen bilgiler girer, kullanıcıdan da girdi alınır. İçlerinde bilimsel karar verme modelleri bulunur. Hoca, herkesin her şeye "yapay zekâ" dediği bugünlerde konuya şu açıdan bakılabileceğini söyledi: Karar destek tarafından bakınca makine öğrenmesi, tahminleme, projeksiyon, sınıflama ya da kümeleme yapan algoritmalarla karar vermenize destek olan çıktılar üretir.

Bu sistemlerin ortak bileşenleri şunlardır:

- Genellikle özelleştirilmiş bir betik ya da programlama dili.
- İstatistik, optimizasyon, makine öğrenmesi, olasılık ve finansal analiz yöntemlerinden en az biri.
- Mutlaka bir veri tabanı.
- Grafik arayüzler: bar ve pasta grafikler, çizgi grafikler, nokta dağılımları. Hoca, borsa haberlerinde görülen, bazen kırmızı bazen yeşil çizgileri olan, üstte ve altta çentikleri bulunan piston benzeri hisse grafiğini de örnek verdi.

  Ek bilgi (derste anlatılmadı): Hocanın tarif ettiği grafik büyük olasılıkla mum grafiğidir (candlestick chart). Bu grafikte her mumun gövdesi büyük olasılıkla bir dönemin açılış ve kapanış fiyatını, üstteki ve alttaki çizgiler de o dönemin en yüksek ve en düşük fiyatını gösterir. Renk de büyük olasılıkla fiyatın o dönemde yükselip düştüğünü belirtir.

Temel çıktılar raporlama, tahminleme, hedef arama (bir arama uzayındaki uygun hedefi bulma) ve optimizasyon çalışmalarıdır.

4.5 Uzman Sistemler

Karar destek sistemi sonucu üretir ama kararı size bırakır: "Bu olabilir" der; kullanırsınız, kullanmazsınız ya da değiştirerek kullanırsınız. Hocaya göre uzman sistem bunun otomatikleştirilmiş hâlidir ve özellikle proses otomasyonunda çok kullanılır.

Hocanın örneği bir üretim hattıdır. Ham madde belli bir seviyenin altına düşer ve tazelenemezse:

1. Üretim hattı yavaşlatılır.
2. İlgili sipariş birimlerine alarm verilir.
3. Belli bir süre içinde yeni besleme yapılmazsa hat durdurulur.
4. Bundan etkilenecek diğer sistemlere bilgi aktarılır.

   Ek bilgi (derste anlatılmadı): Literatürde uzman sistem büyük olasılıkla bir alan uzmanının bilgisini bilgi tabanı ve çıkarım motoruyla taklit eden sistem olarak tanımlanır; tanı koyma ve öneri üretme gibi işler yapar. Bu tanımda kararın otomatik uygulanması büyük olasılıkla zorunlu değildir. Hocanın anlattığı otomatik proses kontrolü bu sistemlerin bir kullanım biçimi olarak düşünülebilir.

4.6 Üst Yönetim Bilgi Sistemleri

Üst yönetim bilgi sistemleri stratejik seviyede çalışır ve yalnız işletme içinden değil işletme dışından da veri alır; yönetim bilgi sistemlerinden ve karar destek sistemlerinden beslenir. En çok yapılan iş veri madenciliğidir. OLAP küpleri ve drill-down analizleri bu sistemlerde sık görülür. Günümüzde makine öğrenmesi, yapay zekâ, tahminleme ve projeksiyon yöntemleri de birlikte kullanılmaktadır. Grafikler, arayüzler ve dashboard denen yapılar çok yaygındır.

> **Ek bilgi (derste anlatılmadı):** OLAP (Online Analytical Processing) büyük olasılıkla çok boyutlu veriyi çözümlemeye yarayan bir sorgulama yaklaşımı olarak bilinir. OLAP küpü büyük olasılıkla ölçüleri zaman, ürün, bölge gibi boyutlar boyunca özetler. Drill-down da büyük olasılıkla özetten ayrıntıya inmeyi anlatır; yıllık özetten aylık ayrıntıya geçmek gibi.

Üst yönetici işletmesinin performansını takip etmek, rakiplerin faaliyetlerini de çözümlemek ister. Bunun için pazar verisine ve bilgisine ihtiyaç duyar. Borsalar ve farklı mecralar gibi dış kaynaklardan toplanan verilerle işletme kendini rakipleriyle kıyaslamaya çalışır. Sistem sorunların ortaya çıkarılmasını ve risk analizini kolaylaştırır. Fırsat değerlendirmesi de yapar: Dışarıdan veri alıp 5 yıllık projeksiyon yaparak "Şu sektördeki şu ürünleri üretmek daha kârlı olacak, bunlar daha çok satacak" gibi çıktılar üretir. Temel çıktılar raporlar ve ileriye yönelik projeksiyonlardır.

Bu tür sistemler doğal olarak daha az sayıdadır ve genellikle şirket, holding ya da iştirak içinde (in-house development) geliştirilir. Bu ölçeğe gelmiş bir işletmenin zaten kendi bilgi işlem departmanı ve üretim mutfağı vardır. Piyasada genel raporlama ve dashboard araçları bulunabilir, ama stratejik seviyede projeksiyon ve analiz yapan hazır bir üst yönetim bilgi sistemi bulmak kolay değildir.

4.7 Bilgi Tabanlı İş Sistemleri

Hoca sona kalan bu türü kısaca toparladı. Bu sistemler, seviye şemasında yeşil ile mavi arasında bir yerde durur. Yalnız veri girdisiyle değil, bilgiyle de çalışırlar. Örnekler CAD/CAM sistemleri, iş istasyonlarındaki finansal analiz uygulamaları, haber verilerini borsa verisine katarak daha anlamlı tahminler yapan uygulamalar ve sanal gerçeklik uygulamalarıdır.

5. Türlerin Karşılaştırılması ve Aralarındaki Etkileşim

5.1 Özet Tablo

Hoca türlerin işlem, çıktı ve kullanıcı farklarını slayttaki bir tabloyla özetledi. Altyazıda okunabilen kısımlar şöyledir:

Tür	Temel işlemler	Kullanıcılar
Üst yönetim bilgi sistemi	Grafikler, simülasyonlar, etkileşimler, analizler	Üst yönetim
Karar destek sistemi	Simülasyonlar ve çeşitli analiz uygulamaları	Yöneticiler
Yönetim bilgi sistemi	Raporlama, temel düzeyde modelleme, işletme ya da birim ölçeğinde düşük düzeyde analiz	Yöneticiler
Bilgi tabanlı iş sistemi	Modelleme, simülasyon, yapay zekâ uygulamaları, artırılmış ve sanal gerçeklik uygulamaları	Uzman ekipler ve teknik personel
Ofis otomasyon sistemi	Ofis işlemleri	İşletmede herkes; özellikle sekreter, muhasebeci, alt kademe yönetici
İşlemsel bilgi sistemi	Veri üzerinde her türlü işlem ve değişiklik	İşletim personeli ve birinci kademe yöneticileri olan şefler

5.2 Etkileşim Şeması

Hoca, türler arasındaki ilişkiyi halkalar ve oklarla gösterilen bir şemayla anlattı. Şemada beş halka vardır, çünkü ofis otomasyon ve bilgi tabanlı iş sistemleri tek halkada gösterilmiştir. Oklar ters yönden de okunabilir: "A, B'ye çıktı üretir" demek, "B, A'dan beslenir" demektir.

Tür	Beslendiği yerler	Çıktı verdiği yerler
İşlemsel bilgi sistemi	Ortamdan veri alır; başka bir bilgi sisteminden veri alması beklenmez	Karar destek, yönetim bilgi, bilgi tabanlı iş ve ofis otomasyon sistemleri
Bilgi tabanlı iş sistemi	İşlemsel bilgi sistemi; gerektiğinde yönetim bilgi sistemi	Yönetim bilgi sistemi, karar destek sistemi
Yönetim bilgi sistemi	Alt katmandaki üç tür	Karar destek ve üst yönetim bilgi sistemleri
Karar destek sistemi	Yönetim bilgi, bilgi tabanlı iş ve işlemsel bilgi sistemleri	Yalnız üst yönetim bilgi sistemi
Üst yönetim bilgi sistemi	Yönetim bilgi ve karar destek sistemleri	Başka bir sisteme çıktı üretmez; yalnız üst yönetime verir

İşlemsel bilgi sistemi "herkese" çıktı üretir, ama üst yönetime doğrudan vermez. Bu bilgi mutlaka yönetim bilgi sistemi ya da karar destek sistemi üzerinden süzülerek ulaşır.

6. Karma Sistemler

Türleri yaptıkları işe, hedef kitlelerine, girdilerine ve çıktılarına göre ayırdık. Ama arada kalmış ya da hibrit sistemler de olabilir. Hoca iki örnek verdi.

Kestirimci bakım örneği. Üretim yapan bir makineyi kontrol eden işlemsel bilgi sistemi, sensörlerden topladığı verilerle hata olup olmadığını izliyor olsun. Burada "şu seviyeden yüksekse alarm üret" gibi çok basit bir kural kullanılabilir. Ama kestirimci bakım (predictive maintenance) yapılarak geçmiş veriler de işlenebilir: Değer yalnız artıyorsa sorun olmaz, ama böyle bir dalgalanma hataya yol açabilir diye öğrenilen bir örüntü alarma dönüştürülebilir. Bu sistemi bilgi tabanlı iş sistemi olarak mı sınıflandıracağız? Hayır. Sistem yine işlemsel bilgi sistemidir, ama içinde yapay zekâ uygulaması barındırır.

Ders öneren öğrenci bilgi sistemi örneği. Öğrenci bilgi sistemi, önceki dersleri ve başarı durumunu inceleyip "Şu seçmeli ders senin için uygun, ister misin?" diye öneri yapsaydı, içinde karar destek unsurları barındırırdı. Temeli yine işlemsel bilgi sistemi olurdu.

> **Hocanın vurgusu:** Bir sistem illa saf bir tür olmak zorunda değildir; farklı türlerin özelliklerini barındırabilir. Ama onu sistemin geneli hangi türdeyse o kategoride tutmayı tercih ediyoruz.

7. Bilgi Sisteminin Tarafları

Bir bilgi sisteminin tarafları kullanıcılar, yöneticiler, sistem analistleri, sistem tasarımcıları, yazılımcılar ve destek personelidir.

7.1 Kullanıcı

Kullanıcı kavramı müşteriyle eşittir ve sistemin varoluş sebebidir. Hoca bunu "müşteri velinimettir" sözüyle anlattı: Sistemi o kullanacak, sistem ona hizmet ediyor, sistem zaten onun için var. Bu yüzden kullanıcı odaklı olmak ve kullanıcının işlerini kolaylaştırmak, bütün özellik ve işlevleri tasarlarken göz önünde bulundurulması gereken temel noktadır.

Kullanıcının sistemden beklentileri net ve tam belirlenmelidir ki doğru ürün üretilebilsin. Bunu belirlemek özellikle sistem analistinin görevidir ve biraz da deneyimle oluşur. Ama yalnız beklentileri belirlemek yetmez: Bir sistemin hangi koşullarda başarılı sayılacağını gösteren ölçüt-

ler ve metrikler de tespit edilmeli, kullanıcı da bunları onaylamalıdır. Kullanıcı "Evet, bu benim için uygun" ya da "Bu ihtiyacımı karşılamaz, en az şu kadar olmalı" diyebilmelidir.

Ders kaydı örneği. Öğrenci bilgi sisteminiz için "Saniyede 3 bin ders kaydı sorunsuz yapılabilir" dediniz. Sistem bu ölçüte göre başarılı olabilir. Ama 10 bin öğrenciniz olduğunu ve herkesin 5'er ders seçeceğini düşünün:

- **Verilen:** 10.000 öğrenci, öğrenci başına en az 5 ders, saniyede 3.000 kayıt.
- **Toplam kayıt:** $10000 \times 5 = 50000$ ders seçimi.
- **Hocanın sonucu:** Bu seçimlerin belli bir süre içinde yapılması gerekir ve belki de bu başarı ölçütü sizin için yeterli değildir. Burada kullanıcı devreye girip "Hayır, bunun şöyle olması lazım" diyerek başarı ölçütünü net biçimde tanımlamalı ya da buna yol göstermelidir.

  Ders sonrası not: Hoca gereken süreyi söylemeden bıraktı. Saniyede 3.000 kayıtla 50.000 kayıt $50000/3000 \approx 16{,}7$ saniyede biter (ara adım eklendi). Yani ölçütün yeterli olup olmadığı, istenen süreye ve seçimlerin zamana nasıl yayıldığına bağlıdır.

Kullanıcı açısından sistem bir **kara kutudur**. Kullanıcı teknik ayrıntılara gömülmek istemez; sistemi Java, .NET, Python ya da başka bir dille yazmış olmanızla ilgilenmez. Arkada ne kodlar yazdığınızı anlatmanıza gerek yoktur. Onun istediği kullanım kolaylığı, doğruluk ve işlevlerin tam olarak yerine getirilmesidir.

> **Hocanın vurgusu:** Tasarım yaparken müşterinin beklentilerini ve başarı ölçütlerini düzgün biçimde anlamak, bilgi sisteminin başarıya ulaşması için çok önemlidir.

7.2 Yönetici

Yönetici rolü ikiye ayrılır. Hoca aklımızda kalsın diye birine "bizden biri", diğerine "patron tayfası" dedi.

Proje yöneticisi projeyi yöneten kişidir ve genellikle teknik kökenlidir. Proje yapa yapa tecrübe kazandıkça kod yazmak ve teknik ayrıntılarla uğraşmak yerine projeyi yönetme tarafına geçer; daha az tecrübeli çalışanların işleri hangi sırayla ve nasıl yapacağını sevk ve idare eder. Projenin başarıya ulaşmasını sağlar. Bunun için kaynakları doğru planlar: en önemlisi insan kaynağı, ayrıca zaman planı ve işlerin sırası. Deneyimine dayanarak gerektiğinde planda değişiklik yapabilmelidir.

Üst düzey yönetici sponsordur. Projenin yapılması için insan, sistem, ağ ve zaman gibi çeşitli kaynaklara ihtiyaç vardır. Bunların hepsi ortak bir değer cinsine çevrilebilir: para ya da daha az kaba söylersek bedel. Bu bedeli karşılayacak tarafa üst düzey yönetici ya da sponsor deriz. Sistemin hangi müşteri ya da iş birimi için yapıldığını destekler ve "Bunun parasını ben veriyorum" diyen taraftır.

> **Hocanın vurgusu:** Üst düzey yönetici için bir projedeki en önemli şey maliyet-fayda analizinin düzgün yapılmasıdır. Hocanın ifadesiyle, attığı taşın mümkün olduğunca çok kuşu ürkütmesini ister; minimum parayla maksimum ürün bekler.

7.3 Sistem Analisti

Sistem analisti dersin odak noktası ve projelerin başarısı için kilit üyedir. Bir yandan teknik bilgisi olmalı, bilgi sistemi terminolojisine ve ayrıntılarına hâkim olmalıdır. Bir yandan da işletme yönetimini bilmelidir: İşlerin nasıl yapıldığını bilmeli ya da kolayca anlayabilmeli; işletmenin yapısını, ihtiyaçlarını, sorunlarını ve iş akışlarını belirleyebilmelidir.

Yaptığı iş köprü olmaktır: İş birimi, yani problemin sahibi ile problemi teknik olarak çözecek roller arasında tercümanlık yapar.

Teknik ekip ve çiftçi örneği. Bir tarafta zehir gibi bir teknik ekip var: React, Flutter, MongoDB, Selenium testleri, hepsini biliyorlar. Hoca bu teknolojileri bilerek saydığını, öğrenciler merak edip araştırsın diye söylediğini de belirtti. Öbür tarafta işini çok iyi bilen, ama bilgi sistemi terminolojisinden anlamayan bir iş birimi var: inşaat, torna, cıvata, tencere-tava ya da tarım. Çiftçi "Toprağı şöyle sürüyoruz, şöyle gübre veriyoruz" diye anlatıyor. Siz ışık miktarı, su miktarı ve topraktaki bileşiklerin analiziyle hangi gübrenin ne kadar verileceğini optimize etmek istiyorsunuz. Ama iki taraf birbirini anlamıyor.

Sistem analisti, ihtiyacı anlayıp teknik dile çevirir. Örneğin şöyle der:

- Birim zamanda ne kadar alana ne kadar su verildiğini ölçmemiz lazım, bunun için su akışını ölçen bir sensör ya da debimetre gerekir.
- Toprakta belli aralıklarla nem sensörleri kullanıp değerleri belli periyotlarla almalı, belirli aralıkların içinde olup olmadığını takip etmeliyiz.
- Aralıkların dışına çıkıldığında sorun oluyor, bu nedenle alarm üretebilmeliyiz.

Bunu duyan teknik tarafta hemen fikirler uçuşmaya başlar: şu sensörü koyarız, şu protokolle verileri kablosuz olarak merkeze alırız, hat koparsa şöyle yaparız, veri tabanını şöyle, arayüzü böyle tasarlarız, şu algoritmayı kullanırız. Hocanın "tercüman" dediği rol tam olarak budur.

Analist, uygulamanın nasıl olacağına ve hangi işlevlere sahip olacağına da kendi başına ya da ekip olarak karar verebilmelidir. Problemi hızla anlamalı, belli bir metodolojiye uygun olarak bilgi sisteminin en azından taslağını oluşturabilmelidir.

Analistin izlediği yol:

1. Problemi araştırır ve anlar.
2. Çözüm için maliyet-fayda analizi yapar.
3. Çözüm için nelerin gerekli olduğunu belirler.
4. Tek bir çözüm üretmez, alternatifler ortaya koyar ve makul bir gerekçeyle en iyisini seçer.
5. Seçtiği çözümün ayrıntılarını ortaya koyar.
6. Çözümün uygulanmasını sağlar.
7. Sonuçların doğru olduğundan ve istekleri karşıladığından emin olmak için izlemeye devam eder.

Analistte aranan beceriler:

- **Analitik düşünme:** Hocaya göre bu her mühendiste olmalıdır, ama analist biraz daha açık göz olup farklı sektörlerle de konuşabilmelidir. Problemi tanımlamalı, sistemi parçalarına ayırıp modüler yapıyı çözümlemeli, hangi parçaların neden ilişkili olduğunu ortaya koymalı ve alternatif çözüm önerebilmelidir.
- **Teknik bilgi:** Neden şu framework olur da öbürü olmaz, neden şu dil (kütüphaneleri iyi olduğu için) kullanılır da bu dil (bir şeyi desteklemediği için) kullanılmaz, neden belli bir haberleşme protokolü seçilir (ucuz olduğu için) ya da ondan kaçınılır (karmaşık olduğu ya da o ortamda çalışmadığı için), bunları bilmelidir. Farklı teknolojilerin nasıl birlikte kullanılıp entegre edileceğini ve geliştirme sürecinin tamamını bilmelidir.
- **Yönetim ve iş bilgisi:** Bir işin nasıl yapıldığına ve organizasyon yapısına hâkim olmalı; sektörü, hedefleri, stratejileri, planları, kurum kültürünü ve değerleri bilmelidir. Proje yöneticisine yardımcı olacağı için kaynak, proje, risk ve değişim yönetimi konularında da bilgi sahibi olmalıdır.
- **İyi insan ilişkileri:** Müşteriyi dinlemeli, ihtiyaçlarını ve işi nasıl yaptığını belirleyebilmeli, davranışlarından sonuç çıkarabilmelidir. Rahat iletişim kurmalı ve paylaşımcı olmalıdır.

İşletmedeki çalışanların bilgi ve beceri seviyesini de bilmelidir. **Depo örneği:** Depodaki çalışanlar RFID'li bir teknolojiyi kullanabilecek seviyede olmayabilir. Bu durumda "barkodu okut, geç" diyen çok daha basit bir barkod çözümü önerilmelidir. Bu örnek ders sonunda bir öğrencinin sorusuna da konu oldu (Bölüm 10.3).

Analist bütün ilerlemeleri belgelemeli ve kayıt altına almalıdır. Çünkü yarın daha iyi bir iş teklifi alıp ayrılırsa projeyi başka bir analist devam ettirecek, sıfırdan başlanmamalıdır. Bu bilgileri diğer paydaşlara da aktarabilmelidir.

Hoca anlattıklarının "Süpermen gibi bir şey" olduğunu kabul etti. Bilgisayar mühendisliği açısından analistliğe sık sık "Ben fıstık gibi kodumu yazarım" diye yaklaşılır. Oysa analist işletmeye, yöneticilere ve teknik tarafa yakındır. Teknik at gözlüğüyle bir yerde kalmak yerine farklı taraflarla etkileşime girdiği için yöneticilik açısından daha uygun bir aday olur.

> **Hocanın vurgusu:** Şirketlerde ara ve üst yönetim kademelerine genellikle sistem analistleri yükselir. "Sistem analisti deyip geçmeyin."

7.4 Tasarımcılar

Tasarımcılar, analistin sunduğu çözüm önerisi ve problemin teknik tarafın anlayacağı modeliyle çalışmaya başlar. Kendi alan bilgileri ve deneyimleriyle çözüm önerileri geliştirir, "Böyle yapacağız" kurgusunu oluştururlar. Alt rolleri şunlardır:

- **Veri tabanı tasarımcıları ve yöneticileri:** Veriyi ve veri tabanını tasarlar ve yönetir.
- **Ağ mimarları (network architects):** Haberleşme protokollerini ve altyapılarını tasarlar.
- **Web ve mobil mimarlar:** Web ve mobil arayüzleri tasarlar.
- **Grafik tasarımcılar:** Grafikleri ve görselleri hazırlar.
- **Güvenlik uzmanları:** Siber güvenlik ve bilgi sistemleri güvenliğiyle çalışır, tasarımın nasıl yapılacağına ve mevcut tasarımların nasıl değiştirileceğine dair girdi sağlar.
- **Özel teknoloji uzmanları:** Özel bir teknoloji kullanılıyorsa onun ayrı uzmanı vardır.

7.5 Yazılım Mimarı ve İşlevsel Olmayan Gereksinimler

Yazılım sistemini tasarlayan kişiye ayrı bir rol verilir: **yazılım mimarı** (software architect). Hoca bu rolü anlatırken gereksinim ayrımını da yaptı.

- **İşlevsel gereksinimler:** Sistemin yapması gereken işlerdir. Bunları yapmazsan bilgi sistemi olamazsın, çünkü müşteri bunları bekliyor.
- **İşlevsel olmayan gereksinimler:** İşlev değil teknolojik beklentilerdir ve genellikle performansa yöneliktir. Örneğin "Dakikada 15 bin ders kaydı yapılabilmeli" ya da "Saniyede şu kadar milyon satır rapor üretilebilmeli".

Yazılım mimarı yazılımın neleri yapacağıyla değil, bunları nasıl, hangi altyapılar üzerinde ve hangi teknolojik kurgularla yapacağıyla ilgilenir. Yazılım mimarilerine hâkim olmak temel sorumluluğudur. Diğer kısıtları karşılayabilmek için ağ ve donanım kaynaklarını, veri kaynaklarını ve veri yönetimini de bilmeli, bu konularda deneyim sahibi olmalıdır. Kodu yazan ekiplerin üstünde, altyapının (infrastructure) nasıl kurgulanacağını, bileşenlerin neler olacağını, nasıl dağıtılacağını, nasıl ayağa kaldırılacağını ve nasıl haberleşeceğini tasarlar ve ekibi yönlendirir.

7.6 Yazılımcılar

Yazılımcılar sistemi kodlar: iş süreçlerini ve problemin gereksinimlerini bilgisayar diline çevirir, analistin girdilerini kullanır ve sistemin çalışan hâlini üretirler. Günümüzde CASE araçları ve geliştirme ortamları (IDE) kod üretimini kolaylaştırmıştır. Kaliteli bir yazılımcı optimize, entegre edilebilir, kolay taşınabilir, değiştirilebilir ve dokümante edilmiş kod yazar.

Yazılımcı şemsiyesinin altında şu roller de değerlendirilir:

- **Sistem programcıları:** Destek sistem programlarını kurar, ayağa kaldırır ve yapılandırır.
- **Veri tabanı programcıları:** Veri tabanında çalışacak kodları ve fonksiyonları yazar. Hocaya göre veri tabanında fonksiyon çalıştırmak çeşitli sebeplerle hâlâ tercih edilebiliyor.
- **Test personeli:** Bütün testlerin yapılmasından sorumlu mühendislerdir.

7.7 Destek Personeli

Destek personeli, sistemin sürekli çalışır hâlde kalmasını ve hizmet sunmasını sağlar. Sorumlulukları şunlardır:

- Ağ iletişimini kontrol etmek, ayakta tutmak, hatayı düzeltmek, düzeltemiyorsa ilgililere haber vermek.
- Donanımı izlemek, gerekiyorsa donanım değişikliklerini yapmak.
- Yazılımı ve ilgili parametreleri kontrol etmek ve düzenlemek, varsa çıktıları düzenlemek.
- Ürün desteği sağlamak: Kullanıcı bir şeyi yapamıyorsa ona nasıl yapılacağını göstermek.
- Güvenlikle ilgili izleme ve işlemleri yapmak.
- Web arayüzlerini izlemek, hata olduğunda ilgili birimlere bildirmek.
- Entegrasyonların sürekliliğini sağlamak.

8. Bilgi Sistemi Geliştirme Süreci

Bilgi sistemi geliştirme süreci, sistemi geliştirmek için izlediğimiz süreç ve uyduğumuz metodolojidir; hocanın sözleriyle yol yürüme tekniğidir. Adımlar ayrıştırılabilir ya da birleştirilebilir, kaynaklarda sayıları değişebilir, ama genel akış şöyledir:

1. **Problem tanımı:** Problem ortaya konur, ne olduğu anlaşılır, ihtiyaçlar kabaca belirlenir.
2. **Fizibilite (yapılabilirlik etüdü):** Projenin kapsamı ve hedefleri ortaya konur. Çıktısı bir rapordur: "Şu şartlar altında yapılabilir; bunun için şunlara ihtiyaç vardır."
3. **Analiz:** Yapılabiliyorsa analize geçilir. İş anlaşılıp teknik tarafa tercüme edilir. Neler yapılacağı ve çözüm yolları ortaya konur, çözüm önerisi ve çözümün mantıksal modeli oluşturulur.
4. **Tasarım:** İkiye bölünebilir.
   - **Genel tasarım:** Sistemin nasıl gerçekleştirileceği belirlenir. Fizibiliteden de destek alınarak maliyet ve üst düzey yapı ortaya konur.
   - **Ayrıntılı tasarım:** Genel tasarımdaki ölçütlere uygun olarak bütün alt sistemler tasarlanır: yazılım modülleri, veri tabanı, ağ ve haberleşme protokolleri, girdi ve çıktı yapıları. Bütün parçaların teorik çözümü hazır hâle gelir.
5. **Gerçekleştirme:** Programlar yazılır, donanım ayarları ve yapılandırmalar yapılır, yükleme ve test aşamalarından geçilir. Sonunda çalışan bir sistem ve ona ait dokümantasyon elde edilir.
6. **Bakım:** Sistem, yaşam döngüsü boyunca bu safhada kalır. Bakımı destek faaliyetleri çerçevesinde yapılır ve sistem çalışır hâlde tutulur.

8.1 Yaşam Döngüsü

> **Hocanın vurgusu:** Bu akışı tek yönlü bir gidişat olarak düşünmemek lazım. Bu sürekli bir döngüdür.

Hocanın hazır aldığı slaytta adımlar biraz farklı adlandırılmıştır: kapsam ve planlama, gereksinim analizi, tasarım, gerçekleştirme, test, kurulum ve dağıtım (devreye alma), operasyon ve bakım. Sonrasında yeni istekler doğabilir, hatalar oluşabilir, zamanla iş yapış şekli değişebilir. O zaman yeniden kapsam ve planlama, analiz, tasarım, test, devreye alma ve operasyon gelir. Yaşam döngüsü birbirini izleyen bir halka olarak sürer.

Sistem artık kullanılmayacaksa döngü bir okun ortasından kırılır. Hocanın ifadesiyle sistemi "öldürdük, gömdük" ya da "rafa kaldırdık" diyerek durursunuz.

9. Süreç Modelleri

Bu adımlar farklı yol yürüme yöntemleriyle yürütülür ve bunun için çok sayıda süreç modeli tanımlanmıştır. Ders kapsamında hepsine eğilinmedi; şunlar kısaca tanıtıldı: klasik süreç (şelale, waterfall) ve varyasyonları; evrimsel süreçler (artımlı ve spiral); Rational Unified Process (RUP); aykırı programlama ve çevik süreçler.

> **Hocanın vurgusu:** "Artık RUP mu kaldı, waterfall mu kaldı? Hep çevik kullanıyoruz" demeyin. Bazı durumlarda şelale gerekebilir, bazı durumlarda çevik süreçler daha iyidir. Hepsinin artıları ve eksileri vardır. Bilgi sisteminin ve projenin yapısına göre kıyas yaparak uygun olan model seçilmelidir.

9.1 Şelale (Waterfall) Modeli

Adı şeklinden gelir: Süreçler yukarıdan başlar ve yer çekimi etkisiyle şelale gibi aşağı doğru düşer. Adımlar birbirini izler, geri dönüş yoktur; trafik tek yönlüdür. Bu yüzden şelale modeli ya da klasik süreç denir.

Soru: Güncellemeler de bakıma dahil mi?

Cevap: Evet, güncelleme bakım aşamasında yapılan çalışmalardan biridir. Konuya ilgili haftada, yaklaşık 8-9 hafta sonra dönülecek.

Model 1970'lerde ortaya çıkmıştır ve bilgisayar mühendisliğinden doğmamıştır; inşaat mühendisliğinden mirastır. Hoca bunu inşaat süreciyle anlattı: Önce zemin etüdü yapılır. Etüde göre

gereksinimler belirlenir ve planlama yapılır. Tasarım gerçekleştirilir. Sonra temel kazılır ve atılır, katlar çıkılır, duvarlar ve camlar örülür, iç tesisat yapılır. Bunlar test edilir, insanlar binayı kullanmaya başlar ve bakım aşaması gelir. "Katı yanlış attık, son üç katı kesip yeniden yapalım" diye bir şey söz konusu değildir. Bu yüzden her adım tam olarak bitirilip sonrakine geçilir.

> **Ek bilgi (derste anlatılmadı):** Model büyük olasılıkla Winston Royce'un 1970 tarihli makalesine dayandırılır. Royce'un bu makalede saf sıralı işleyişi riskli bulduğu ve geri besleme döngüleri önerdiği de büyük olasılıkla doğrudur; bu bakımdan aşağıdaki geri dönüşlü varyasyon erken bir tartışmayla ilişkilidir.

Bilgisayar mühendisliğinde şelale kısa biçimiyle gereksinim analizi, tasarım, gerçekleştirme, test ve bakımdır. Daha ayrıntılı biçimi problem tanımı ve gereksinimlerin toplanması, ön inceleme ya da fizibilite, analiz, tasarım, kodlama, test, devreye alma ve bakımdır. Adımları çoğaltabilir ya da az tutabilirsiniz; önemli olan genel adımları aşağı doğru, geri dönmeyecek biçimde işletmektir.

9.2 Şelalenin Varyasyonları

Geri dönüşlü (iteratif, artımlı) şelale: Gereksinim analizinden tasarıma, tasarımdan gerçekleştirmeye, oradan devreye alma ve bakıma geçilir, ama farklı aşamalara geri dönülebilir. "Şurada eksik var, fazla var, hata var" denip akış oradan sürdürülür.

Prototip modeli (model oluşturma): Gereksinimler derlenir, taslak bir tasarım ve ondan bir model oluşturulur. Prototip yeni özellikler ve gereksinimler eklenerek sürüm sürüm (versiyon 0, 1, 2, 3) genişletilir. Sonrasında gereksinimler işlenip gerçekleştirilerek yazılım üretilir. Burada daha devingen ve adım adım ilerleyen bir yapı vardır.

Hoca bunları yalnız tanıtmak için anlattığını, asıl önemli olanın şelale olduğunu belirtti. Merak edenler "yazılım süreç modelleri" diye araştırabilir, ayrıca Yazılım Mühendisliği dersinde bu konular üzerinde daha fazla durulacak.

9.3 Evrimsel Süreçler: Artımlı ve Spiral Model

Artımlı model: Şelale iteratif olarak uygulanır. Bir kez uygulanıp bir ürün elde edilir, ikinci şelale ürünü geliştirir, üçüncüsü daha da geliştirir.

Spiral model: Hocanın dediğine göre bir ara çok moda olmuştur. Altı faz vardır ve spiralin her dönüşünde bu fazlardan yeniden geçilir:

1. **Müşteri iletişimi:** Gereksinimler ve problem tanımı.
2. **Planlama:** Proje planı.
3. **Risk analizi:** Risk güdümlü çalışılır. Neyin sıkıntı yaratabileceği belirlenir ve risk yaratabilecek kısımlar öne alınıp önce gerçekleştirilir.
4. **Mühendislik:** Analiz ve tasarım.
5. **Gerçekleştirme:** Gerekiyorsa güncellemeler ve testlerle birlikte.
6. **Müşteri değerlendirmesi:** Buradan gelen girdilerle yeni tur başlar.

Spiral içeriden başlar. Eksenler boyunca kavram geliştirme, yeni ürün geliştirme, ürün zenginleştirme ve ürün bakımı süreçleri gelir. Ürün prototipten çalışan son sürüme doğru her dönüşte olgunlaşır.

9.4 Unified Process (RUP)

Hocanın anlatımıyla 2000'lerde, nesneye dayalı programlamanın gelişmesiyle Unified Process öne çıktı. Üç özelliği vardır: Kullanım senaryolarını (use case) ön plana alır, mimari yapıyı merkeze koyar ve artımlı ve iteratif çalışır.

Proje dört ana faza bölünür: başlangıç, geliştirme, üretim ve geçiş. Bu fazların içinde birden çok iterasyon olabilir. Başlangıç tek safha, ayrıntılandırma iki üç safha, üretim n iterasyon olabilir. Hocanın gösterdiği şekilde 4 faz ama 10 safha vardır, yani 10 kez dönülür. Her satır bir iterasyondur. Kutucuklar ise o iterasyonda gereksinim belirleme, analiz, tasarım, gerçekleştirme ve test gibi ana adımlara hangi ağırlıkla zaman harcandığını gösterir.

> **Ders sonrası not:** Hoca fazları "başlangıç, geliştirme, üretim, geçiş" diye saydı, anlatırken "olgunlaştırma" ve "detaylandırma" da dedi. Standart adlar büyük olasılıkla Inception, Elaboration, Construction, Transition'dır; Türkçe karşılıkları başlangıç, olgunlaştırma (ayrıntılandırma), inşa (yapım), geçiş olarak düşünülebilir.

Ağırlıklar şöyle kayar: Başlangıçta en büyük kutu gereksinim belirlemededir, analiz daha azdır, diğerleri neredeyse yoktur. Olgunlaştırmada analiz çok artar, sonra tasarım ağırlaşır. Üretim aşamasında gerçekleştirme bir anda çok yüklenir, test artar. Geçişte test sürer, gerçekleştirme tarafında güncelleme, değişiklik ve hata giderme devam eder. Hocanın özetiyle büyük kutular sol üstten sağ alta doğru akar.

Her fazda ve iterasyonda hangi model çıktısının ne işe yaradığı da bellidir. Kullanım senaryosu modeliyle analiz tanımlanır, sonra tasarım modeli oluşturulur, sonra barındırma modeliyle gerçekleştirmedeki bileşen yapısı çözümlenir, en son test modelinde neyin nasıl test edileceği söylenir.

Hoca, bunun bir zamanlar "5000 sayfalık falan", tuğla gibi kalın bir kitabı olduğunu söyledi. Bugün Unified Process doğrudan pek kullanılmıyor, ama pek çok öğesi yeni süreç modellerine girdi oldu.

9.5 Aykırı Programlama (Extreme Programming, XP)

Hoca, Unified Process'ten sonra aykırı programlama manifestosunun yayımlandığını söyledi.

> **Ders sonrası not:** Sıralama büyük olasılıkla bu kadar kesin değildir: XP de büyük olasılıkla 1990'ların sonunda ortaya çıkmış, Unified Process ile aşağı yukarı aynı dönemde yaygınlaşmıştır.

Dört temel değeri vardır: **iletişim, basitlik, geri besleme ve cesaret**. Bu değerler üzerine 12 pratik önerilmiştir. Hocanın saydığı pratikler şunlardır:

- Sistemin bir metafor olarak ele alınması.
- Ekip üyelerinin müşteri rolü alması, müşterinin işin içinde tutulması.
- Kısa aralıklarla, küçük sürümler hâlinde yayım yapılması.
- Tasarımın bir plan oyunu çerçevesinde üretilmesi ve basit tutulması.
- Kodun bir kişiye değil ekibe ait olması (ortak kod mülkiyeti).
- Kodlama standartlarına her koşulda uyulması.
- Eşli programlama (pair programming).
- Testin ön plana alınması (test güdümlü geliştirme).
- Ürüne sürekli ekleme ve tümleştirme yapılması.
- Sürekli yeniden tasarım ve kod iyileştirme (refactoring).
- Sabit ve sürdürülebilir çalışma temposu.

Eşli programlama. Hocaya göre XP'nin eski süreçlerden en çok ayrıldığı noktalardan biridir. Bir kodu tek kişi yazmaz; iki geliştirici aynı bilgisayarda yan yana oturup tek bir kod üretir. Yazarken aynı anda (on the fly) test eder ve düşünürler: "Sen onu öyle mi yazdın, şöyle yapsak daha iyi olmaz mı?" Ortak akıl daha iyi tek bir ürün çıkarır.

Sürekli tümleştirme. Bugün CI/CD ve DevOps süreçleri dediğimiz şeylerin atası bu pratiktir.

> **Ders sonrası not:** Hoca CI/CD'yi "Continuous Integration, Continuous Development" diye açtı. CD'nin yaygın açılımı büyük olasılıkla Continuous Delivery ya da Continuous Deployment'tır.

Sabit tempo. XP örneğin "Haftada 40 saat çalışın, ama her hafta 40 saat çalışın" der. Hoca $5 \times 8 = 40$ saat örneğini verdi, 6 günlük dağılımı da söyledi. Bu süre içinde verimli çalışılır; ne çok ne az, hep sabit hızla çalışılır, sonra durulur.

> **Ders sonrası not:** Hoca 6 günlük çalışmada günlük süreyi "6 saat mi, 7 saat mi" diye tereddütle söyledi. $6 \times 6 = 36$ ve $6 \times 7 = 42$ olduğu için ikisi de 40 etmez; tam değer $40/6 \approx 6{,}67$ saattir.

Bu yaklaşım çevik manifestoya da uygundur: Müşteriyi işin içinde tutar, artımlı çalışmayı teşvik eder, sürekli test ve sürekli entegrasyon ister. Küçük sürümler konusunda hoca, yazılım

sürümlerinde görülen 1.5.62.33 ya da 5.2.3.47 gibi numaraları örnek verdi. Sondaki en küçük parça derleme numarasıdır (build number) ve kaçıncı kararlı derlemenin kullanıldığını izlemeyi sağlar.

> **Ders sonrası not:** Hoca bu çok parçalı numaraların "tamamen aykırı programlamadan geldiğini" söyledi. Bu büyük olasılıkla basitleştirmedir; sürüm numaralandırma büyük olasılıkla XP'ye özgü değildir. XP'den gelen fikir, küçük ve sık sürüm çıkarmaktır.

XP dokümantasyona çok fazla önem vermez; kullanıcıya uzun uzun kitap yazmanızı istemez. Ama düzgün bir kod standardı açısından kodun dokümante edilmesi gerekir. Hoca XP'yi incelemeyi tavsiye etti: Özellikle yazılım mühendisliği alanında çalışmak isteyenler için modern tekniklerin kaynağını görmek açısından faydalıdır.

9.6 Çevik (Agile) Yöntemler

Aykırı programlamadan sonra çevik süreçler öne çıktı ve köklerini XP'den alan yeni süreç modelleri tanıtıldı. Günümüzde en yaygın olanı Scrum'dır; Kanban ve Lean de biraz daha farklılaştırılmış modellerdir.

> **Hocanın vurgusu:** Bu dersin konusu tam olarak bu modeller değildir. Önemli olan geliştirme sürecindeki adımlarda ne yapıldığı, ne çıktı üretildiği ve bunun sürekli bir döngü olduğudur. Çevik süreçler günümüzde en yaygın olanlardır, ama **klasik süreç yani şelale modeli bu ders kapsamında temel teşkil eder.**

10. Ders Sonundaki Sorular

10.1 Eşli Programlama Bugün Kullanılıyor mu?

Hoca ders kapanışında eşli kodlamaya döndü.

Soru: Eşli kodlama günümüzde devam ediyor mu?

Cevap: Herkes devam ettirmese de bunu sürdüren bazı uygulamalar ve işletmeler hâlâ var.

10.2 Çözüm mü Ekibe, Ekip mi Çözüme Göre?

Soru: Analist çözüm önerirken ekibin niteliklerine göre bir yol izlemeli dediniz. Ama önemli olan müşteriye nitelikli bir çözüm ulaştırmaksa, çözümü ekibe göre değil, ekibi çözüme göre oluşturmak gerekmez mi?

Cevap: Haklısın, ama bunu her zaman yapmak mümkün değil. Bazı işletmelerin bazı birimlerinde kilit elemanlar vardır. Kişi 20 yıldır oradadır, her şeyi o bilir ve değiştirilemez. Bu kişiler de genellikle teknolojiye ve yeniliğe karşıdır: İşini çok kolaylaştıracak bir RFID sistemiyle gelirsiniz, "Ben bunu kullanmam" der. Bunu işletme sahibine anlatırsınız: "Biz şunu öneriyoruz, ama buradaki personel bu teknolojiye hazır değil. Personeli değiştirirseniz bu kullanılabilir. Personel değişmeyecekse eski de olsa ancak şu teknolojiyi önerebiliriz."

Hoca bunu kendi deneyimiyle de örnekledi: Hayatı boyunca terminal kullanmış ve grafik arayüzü görünce kaçan tek bir kişi yüzünden, onun kullanması için terminal ekranından veri girişi yapılan bir ekran geliştirmek zorunda kalmışlar.

Dersin Toparlaması

Ders üç temel başlık üzerinde durdu. Önce bilgi sistemleri amaçlarına göre altı türe ayrıldı: işlemsel seviyede işlemsel bilgi sistemi, bilgi seviyesinde ofis otomasyon ve bilgi tabanlı iş sistemleri, yönetim seviyesinde yönetim bilgi ve karar destek sistemleri, stratejik seviyede üst yönetim bilgi sistemi. Türlerin birbirinden nasıl beslendiği görüldü; karma sistemlerin de genel amaçlarına göre sınıflandırıldığı söylendi. Ardından taraflar ele alındı: kullanıcı, proje yöneticisi ve sponsor, sistem analisti, tasarımcılar ve yazılım mimarı, yazılımcılar, destek personeli. Dersin odağı olan sistem analisti, iş birimi ile teknik ekip arasındaki tercüman olarak tanımlandı.

Son başlıkta geliştirme süreci problem tanımından bakıma uzanan bir döngü olarak anlatıldı ve süreç modelleri şelaleden çevik yöntemlere kadar tanıtıldı. Dersin temel modeli şelaledir. Sonraki derste bu sürecin adımları teker teker işlenmeye başlanacak: ön inceleme, problemin anlaşılması, fizibilite çalışması, ne tür fizibilitelerin ortaya konduğu ve nasıl raporlandığı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.